

Лабораторная работа № 5

**Дискреционное разграничение прав в Linux. Исследование
влияния дополнительных атрибутов**

Мальянц Виктория Кареновна

Содержание

1	Цель работы	6
2	Задание	7
3	Выполнение лабораторной работы	8
3.1	Подготовка лабораторного стенда	8
3.2	Создание программы	9
3.3	Исследование Sticky-бита	16
4	Выводы	20
	Список литературы	21

Список иллюстраций

3.1	Применение команды gcc -v	8
3.2	Применение команд sudo setenforce 0 и getenforce	9
3.3	Применение команды sudo -u guest -i	9
3.4	Создание программы simpleid.c	9
3.5	Код программы simpleid.c	10
3.6	Применение команды gcc simpleid.c -o simpleid	10
3.7	Применение команды ./simpleid	10
3.8	Создание программы simpleid2.c	10
3.9	Код программы simpleid2.c	12
3.10	Применение команд gcc simpleid2.c -o simpleid2 и ./simpleid2 . . .	12
3.11	Применение команд sudo chown root:guest /home/guest/simpleid2 и sudo chmod u+s /home/guest/simpleid2	12
3.12	Применение команды ls -l simpleid2	13
3.13	Применение команд ./simpleid2 и id	13
3.14	Создание программы readfile.c	13
3.15	Код программы readfile.c	14
3.16	Применение команды gcc readfile.c -o readfile	14
3.17	Изменение прав	15
3.18	Проверка после изменения прав	15
3.19	Проверка чтения файла readfile.c	15
3.20	Проверка чтения файла /etc/shadow	15
3.21	Проверка чтения файла /etc/shadow от суперпользователя	16
3.22	Применение команды ls -l / grep tmp	16
3.23	Применение команды echo «test» > /tmp/file01.txt	16
3.24	Применение команд ls -l /tmp/file01.txt, chmod o+rw /tmp/file01.txt, ls -l /tmp/file01.txt	17
3.25	Применение команды cat /tmp/file01.txt	17
3.26	Применение команды echo «test2» > /tmp/file01.txt	17
3.27	Применение команды cat /tmp/file01.txt	17
3.28	Применение команды echo «test3» > /tmp/file01.txt	17
3.29	Применение команды cat /tmp/file01.txt	18
3.30	Применение команды rm /tmp/file01.txt	18
3.31	Применение команды chmod -t /tmp	18
3.32	Применение команды exit	18
3.33	Применение команды ls -l / grep tmp	18
3.34	Повторение предыдущих шагов	19

3.35 Применение команд su -, chmod +t /tmp, exit	19
--	----

Список таблиц

1 Цель работы

Изучение механизмов изменения идентификаторов, применения SetUID- и Sticky-битов. Получение практических навыков работы в консоли с дополнительными атрибутами. Рассмотрение работы механизма смены идентификатора процессов пользователей, а также влияние бита Sticky на запись и удаление файлов.

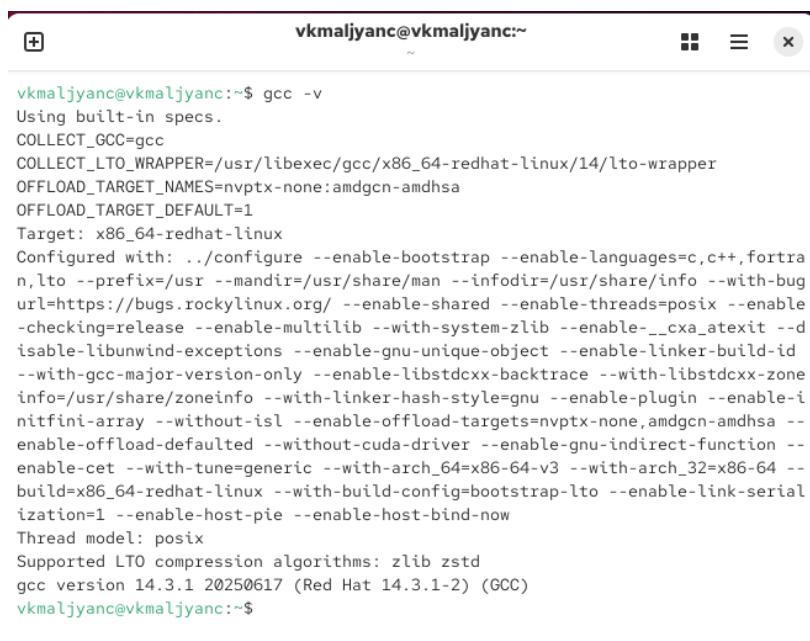
2 Задание

1. Подготовка лабораторного стенда
2. Создание программы
3. Исследование Sticky-бита

3 Выполнение лабораторной работы

3.1 Подготовка лабораторного стенда

Убеждаюсь, что в системе установлен компилятор gcc с помощью команды gcc -v (рис. 3.1).



```
vkmaljyanc@vkmaljyanc:~$ gcc -v
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/usr/libexec/gcc/x86_64-redhat-linux/14/lto-wrapper
OFFLOAD_TARGET_NAMES=nvptx-none:amdgc- amdhsa
OFFLOAD_TARGET_DEFAULT=1
Target: x86_64-redhat-linux
Configured with: ../configure --enable-bootstrap --enable-languages=c,c++,fortran,lto --prefix=/usr --mandir=/usr/share/man --infodir=/usr/share/info --with-bug-url=https://bugs.rockylinux.org/ --enable-shared --enable-threads=posix --enable-checking=release --enable-multilib --with-system-zlib --enable-__cxa_atexit --disable-libunwind-exceptions --enable-gnu-unique-object --enable-linker-build-id --with-gcc-major-version-only --enable-libstdc++-backtrace --with-libstdc++-zoneinfo=/usr/share/zoneinfo --with-linker-hash-style=gnu --enable-plugin --enable-initfini-array --without-isl --enable-offload-targets=nvptx-none,amdgc- amdhsa --enable-offload-defaulted --without-cuda-driver --enable-gnu-indirect-function --enable-cet --with-tune=generic --with-arch_64=x86-64-v3 --with-arch_32=x86-64 --build=x86_64-redhat-linux --with-build-config=bootstrap-lto --enable-link-serialization=1 --enable-host-pie --enable-host-bind-now
Thread model: posix
Supported LTO compression algorithms: zlib zstd
gcc version 14.3.1 20250617 (Red Hat 14.3.1-2) (GCC)
vkmaljyanc@vkmaljyanc:~$
```

Рисунок 3.1: Применение команды gcc -v

Отключаю систему запретов до очередной перезагрузки системы командой sudo setenforce 0. После этого команда getenforce выводит Permissive (рис. 3.2).


```
vkmaljyanc@vkmaljyanc:~$ sudo setenforce 0
[sudo] password for vkmaljyanc:
vkmaljyanc@vkmaljyanc:~$ getenforce
Permissive
vkmaljyanc@vkmaljyanc:~$
```

Рисунок 3.2: Применение команд `sudo setenforce 0` и `getenforce`

3.2 Создание программы

Вхожу в систему от имени пользователя `guest` с помощью команды `sudo -u guest -i` (рис. 3.3).

```
vkmaljyanc@vkmaljyanc:~$ sudo -u guest -i
guest@vkmaljyanc:~$ █
```

Рисунок 3.3: Применение команды `sudo -u guest -i`

Создаю программу `simpleid.c` (рис. 3.4).

```
guest@vkmaljyanc:~$ touch simpleid.c
guest@vkmaljyanc:~$ nano simpleid.c
```

Рисунок 3.4: Создание программы `simpleid.c`

Листинг программы `simpleid.c`:

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>

int
main ()
{
    uid_t uid = geteuid ();
    gid_t gid = getegid ();
    printf ("uid=%d, gid=%d\n", uid, gid);
    return 0;
}
```

Код программы simpleid.c (рис. 3.5).



```
guest@vkmaljyanc:~ -- sudo -u guest -i
GNU nano 8.1 simpleid.c
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
int
main ()
{
    uid_t uid = geteuid ();
    gid_t gid = getegid ();
    printf ("uid=%d, gid=%d\n", uid, gid);
    return 0;
}
```

[Wrote 11 lines]

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line

Рисунок 3.5: Код программы simpleid.c

Компилирую программу и убеждаюсь, что файл программы создан: gcc simpleid.c -o simpleid (рис. 3.6).

```
guest@vkmaljyanc:~$ gcc simpleid.c -o simpleid
guest@vkmaljyanc:~$ ls
dir1 simpleid simpleid.c
```

Рисунок 3.6: Применение команды gcc simpleid.c -o simpleid

Выполняю программу simpleid: ./simpleid. Команда id выводит номера пользователя и групп (рис. 3.7).

```
guest@vkmaljyanc:~$ ./simpleid
uid=1003, gid=1003 _
```

Рисунок 3.7: Применение команды ./simpleid

Создаю программу simpleid2.c (рис. 3.8).

```
guest@vkmaljyanc:~$ touch simpleid2.c
guest@vkmaljyanc:~$ nano simpleid2.c
```

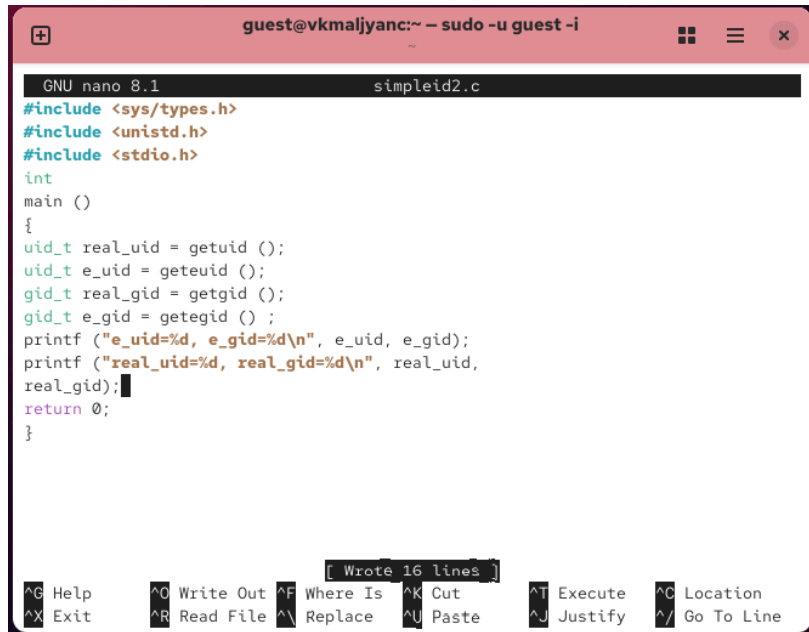
Рисунок 3.8: Создание программы simpleid2.c

Листинг программы simpleid2.c:

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>

int
main ()
{
    uid_t real_uid = getuid ();
    uid_t e_uid = geteuid ();
    gid_t real_gid = getgid ();
    gid_t e_gid = getegid () ;
    printf ("e_uid=%d, e_gid=%d\n", e_uid, e_gid);
    printf ("real_uid=%d, real_gid=%d\n", real_uid,
real_gid);↵↵
    return 0;
}
```

Код программы simpleid2.c (рис. 3.9).

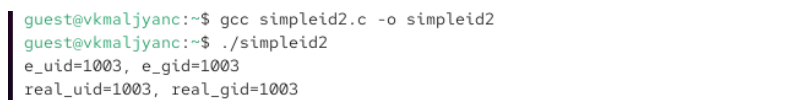
A screenshot of a terminal window with the title bar 'guest@vkmaljyanc:~ - sudo -u guest -i'. The terminal shows the GNU nano 8.1 editor editing a file named 'simpleid2.c'. The code is as follows:

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
int
main ()
{
    uid_t real_uid = getuid ();
    uid_t e_uid = geteuid ();
    gid_t real_gid = getgid ();
    gid_t e_gid = getegid ();
    printf ("e_uid=%d, e_gid=%d\n", e_uid, e_gid);
    printf ("real_uid=%d, real_gid=%d\n", real_uid,
    real_gid);
    return 0;
}
```

At the bottom of the editor, a status bar indicates '[Wrote 16 lines]'. Below the editor, there is a row of keyboard shortcuts: ^G Help, ^O Write Out, ^F Where Is, ^K Cut, ^T Execute, ^C Location, ^X Exit, ^R Read File, ^_ Replace, ^U Paste, ^J Justify, and ^_ Go To Line.

Рисунок 3.9: Код программы simpleid2.c

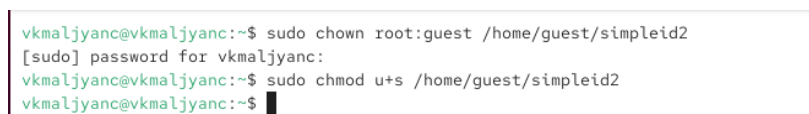
Компилирую и запускаю simpleid2.c. Применение команды gcc simpleid2.c -o simpleid2 и ./simpleid2 (рис. 3.10).

A screenshot of a terminal window showing the following commands and output:

```
guest@vkmaljyanc:~$ gcc simpleid2.c -o simpleid2
guest@vkmaljyanc:~$ ./simpleid2
e_uid=1003, e_gid=1003
real_uid=1003, real_gid=1003
```

Рисунок 3.10: Применение команд gcc simpleid2.c -o simpleid2 и ./simpleid2

От имени суперпользователя выполняю команды: chown root:guest /home/guest/simpleid2 и chmod u+s /home/guest/simpleid2 (рис. 3.11).

A screenshot of a terminal window showing the following commands and output:

```
vkmaljyanc@vkmaljyanc:~$ sudo chown root:guest /home/guest/simpleid2
[sudo] password for vkmaljyanc:
vkmaljyanc@vkmaljyanc:~$ sudo chmod u+s /home/guest/simpleid2
vkmaljyanc@vkmaljyanc:~$
```

Рисунок 3.11: Применение команд sudo chown root:guest /home/guest/simpleid2 и sudo chmod u+s /home/guest/simpleid2

Выполняю проверку правильности установки новых атрибутов и смены владельца файла simpleid2: ls -l simpleid2 (рис. 3.12).

```

vkmaljyanc@vkmaljyanc:~$ sudo -i
root@vkmaljyanc:~# ls -l /home/guest/simpleid2
-rwsr-xr-x. 1 root guest 16920 Apr  2 19:09 /home/guest/simpleid2
root@vkmaljyanc:~# █

```

Рисунок 3.12: Применение команды `ls -l simpleid2`

Запускаю `simpleid2` и `id`: `./simpleid2` и `id`. Вывод команды `id` более подробный (рис. 3.13).

```

guest@vkmaljyanc:~$ ./simpleid2
e_uid=0, e_gid=1003
real_uid=1003, real_gid=1003
guest@vkmaljyanc:~$ id
uid=1003(guest) gid=1003(guest) groups=1003(guest) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
guest@vkmaljyanc:~$ █

```

Рисунок 3.13: Применение команд `./simpleid2` и `id`

Создаю программу `readfile.c` (рис. 3.14).

```

guest@vkmaljyanc:~$ touch readfile.c
guest@vkmaljyanc:~$ nano readfile.c █

```

Рисунок 3.14: Создание программы `readfile.c`

Листинг программы `readfile.c`:

```

#include <fcntl.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
int
main (int argc, char* argv[])
{
    unsigned char buffer[16];
    size_t bytes_read;
    int i;

```

```

int fd = open (argv[1], O_RDONLY);
do
{
bytes_read = read (fd, buffer, sizeof (buffer));
for (i =0; i < bytes_read; ++i) printf("%c", buffer[i]);
}

```

Код программы readfile.c (рис. 3.15).



The screenshot shows a terminal window with two tabs: 'guest@vkmaljiyanc:~ - sudo -u guest -i' and 'root@vkmaljiyanc:~ - sudo -i'. The active tab is the guest user session. The terminal shows the GNU nano 8.1 editor editing 'readfile.c'. The code is as follows:

```

#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
int
main (int argc, char* argv[])
{
    unsigned char buffer[16];
    size_t bytes_read;
    int i;
    int fd = open (argv[1], O_RDONLY);
    do
    {
        bytes_read = read (fd, buffer, sizeof (buffer));
        for (i =0; i < bytes_read; ++i) printf("%c", buffer[i]);
    }
    while (bytes_read == sizeof (buffer));
    close (fd);
    return 0;
}

```

At the bottom of the terminal, a status bar indicates '[Wrote 21 lines]' and a list of keyboard shortcuts: ^G Help, ^O Write Out, ^F Where Is, ^K Cut, ^T Execute, ^C Location, ^X Exit, ^R Read File, ^\ Replace, ^U Paste, ^J Justify, ^_ Go To Line.

Рисунок 3.15: Код программы readfile.c

Компилирую её с помощью команды gcc readfile.c -o readfile (рис. 3.16).

```

guest@vkmaljiyanc:~$ gcc readfile.c -o readfile

```

Рисунок 3.16: Применение команды gcc readfile.c -o readfile

Меняю владельца у файла readfile.c и изменяю права так, чтобы только суперпользователь (root) мог прочитать его, а guest не мог (рис. 3.17).

```

vkmaljyanc@vkmaljyanc:~$ sudo chown root:guest /home/guest/readfile.c
[sudo] password for vkmaljyanc:
vkmaljyanc@vkmaljyanc:~$ sudo chmod u+s /home/guest/readfile.c
vkmaljyanc@vkmaljyanc:~$ sudo chmod 700 /home/guest/readfile.c
vkmaljyanc@vkmaljyanc:~$ sudo chmod -r /home/guest/readfile.c
vkmaljyanc@vkmaljyanc:~$ sudo chmod u+s /home/guest/readfile.c
vkmaljyanc@vkmaljyanc:~$

```

Рисунок 3.17: Изменение прав

Проверяю, что пользователь guest не может прочитать файл readfile.c (рис. 3.18).

```

guest@vkmaljyanc:~$ cat readfile.c
cat: readfile.c: Permission denied

```

Рисунок 3.18: Проверка после изменения прав

Меняю у программы readfile владельца и устанавливаю SetU'D-бит. Проверяю, может ли программа readfile прочитать файл readfile.c (рис. 3.19).

```

guest@vkmaljyanc:~$ ./readfile readfile.c
D##### zD<zDV@#####X#####i###Jlp#####0@zD==@|i6N\\|i2###]X#####
##{I zDV@==@E@zDp@P#####@H####8#####/###C###C####h####v#####
#####μ#####>#####0#####c#####t#####9#####D#####O#####W#####j
|<zD#####p#####f#####!<zD3#####d@8
|
#####$#####{K#a
|Q4x86_64./readfilereadfile.cSHELL=/bin/bashCOLORTERM=truecolorSUDO_GID=1000HIS

```

Рисунок 3.19: Проверка чтения файла readfile.c

Проверяю, может ли программа readfile прочитать файл /etc/shadow (рис. 3.20).

```

guest@vkmaljyanc:~$ ./readfile /etc/shadow
#####< X VepB#####B#####gW#####B`[ ==@&c _xU&};###T##B#c
o\###&I< V@==@u[ p@##B#####@x#B#C##C##C##/C##CC##QC##hC##vC##C##
C##C##C##C##C##>C##OC##cC##tC##C##C##9C##DC##0C##WC##jC##C##C##C##]
X##C##p@##C##!X 3#####d@8
|
#####$#####B##co\###&+##CUx86_64./readfile/etc/shadowSHELL=/bin/bashCOLORTER

```

Рисунок 3.20: Проверка чтения файла /etc/shadow

Проверяю, может ли программа readfile прочитать файл /etc/shadow от суперпользователя (рис. 3.21).


```

guest@vkmaljyanc:~$ ls -l /tmp/file01.txt
-rw-r--r--. 1 guest guest 5 Apr  2 19:30 /tmp/file01.txt
guest@vkmaljyanc:~$ chmod o+rw /tmp/file01.txt
guest@vkmaljyanc:~$ ls -l /tmp/file01.txt
-rw-r--r--. 1 guest guest 5 Apr  2 19:30 /tmp/file01.txt
guest@vkmaljyanc:~$

```

Рисунок 3.24: Применение команд `ls -l /tmp/file01.txt`, `chmod o+rw /tmp/file01.txt`, `ls -l /tmp/file01.txt`

От пользователя `guest2` (не являющегося владельцем) пробую прочитать файл `/tmp/file01.txt`: `cat /tmp/file01.txt` (рис. 3.25).

```

vkmaljyanc@vkmaljyanc:~$ sudo -u guest2 -i
[sudo] password for vkmaljyanc:
guest2@vkmaljyanc:~$ cat /tmp/file01.txt
test

```

Рисунок 3.25: Применение команды `cat /tmp/file01.txt`

От пользователя `guest2` пробую дозаписать в файл `/tmp/file01.txt` слово `test2` командой `echo «test2» > /tmp/file01.txt`. Не удалось выполнить операцию (рис. 3.26).

```

guest2@vkmaljyanc:~$ echo "test2" > /tmp/file01.txt
-bash: /tmp/file01.txt: Permission denied
guest2@vkmaljyanc:~$ █

```

Рисунок 3.26: Применение команды `echo «test2» > /tmp/file01.txt`

Проверяю содержимое файла командой `cat /tmp/file01.txt` (рис. 3.27).

```

guest2@vkmaljyanc:~$ cat /tmp/file01.txt
test
guest2@vkmaljyanc:~$ █

```

Рисунок 3.27: Применение команды `cat /tmp/file01.txt`

От пользователя `guest2` пробую записать в файл `/tmp/file01.txt` слово `test3`, стерев при этом всю имеющуюся в файле информацию командой `echo «test3» > /tmp/file01.txt`. Не удалось выполнить операцию (рис. 3.28).

```

guest2@vkmaljyanc:~$ echo "test3" > /tmp/file01.txt
-bash: /tmp/file01.txt: Permission denied
guest2@vkmaljyanc:~$

```

Рисунок 3.28: Применение команды `echo «test3» > /tmp/file01.txt`

Проверяю содержимое файла командой `cat /tmp/file01.txt` (рис. 3.29).

```
guest2@vkmaljanc:~$ cat /tmp/file01.txt
test
```

Рисунок 3.29: Применение команды `cat /tmp/file01.txt`

От пользователя `guest2` пробую удалить файл `/tmp/file01.txt` командой `rm /tmp/file01.txt`. Не удалось удалить файл (рис. 3.30).

```
guest2@vkmaljanc:~$ rm /tmp/file01.txt
rm: remove write-protected regular file '/tmp/file01.txt'? y
rm: cannot remove '/tmp/file01.txt': Operation not permitted
guest2@vkmaljanc:~$
```

Рисунок 3.30: Применение команды `rm /tmp/file01.txt`

Повышаю свои права до суперпользователя следующей командой `su -` и выполняю после этого команду, снимающую атрибут `t` (Sticky-бит) с директории `/tmp`: `chmod -t /tmp` (рис. 3.31).

```
guest2@vkmaljanc:~$ su -
Password:
Last login: Sat Feb 14 13:43:11 MSK 2026 on pts/0
root@vkmaljanc:~# chmod -t /tmp
```

Рисунок 3.31: Применение команды `chmod -t /tmp`

Покидаю режим суперпользователя командой `exit` (рис. 3.32).

```
root@vkmaljanc:~# exit
logout
guest2@vkmaljanc:~$
```

Рисунок 3.32: Применение команды `exit`

От пользователя `guest2` проверяю, что атрибута `t` у директории `/tmp` нет: `ls -l / | grep tmp` (рис. 3.33).

```
guest2@vkmaljanc:~$ ls -l / | grep tmp
drwxrwxrwx. 14 root root 4096 Apr  2 19:39 tmp
guest2@vkmaljanc:~$
```

Рисунок 3.33: Применение команды `ls -l / | grep tmp`

Повторяю предыдущие шаги. Удалось удалить файл (рис. 3.34).

```

guest2@vkmaljyanc:~$ cat /tmp/file01.txt
test
guest2@vkmaljyanc:~$ echo "test2" > /tmp/file01.txt
-bash: /tmp/file01.txt: Permission denied
guest2@vkmaljyanc:~$ cat /tmp/file01.txt
test
guest2@vkmaljyanc:~$ echo "test3" > /tmp/file01.txt
-bash: /tmp/file01.txt: Permission denied
guest2@vkmaljyanc:~$ cat /tmp/file01.txt
test
guest2@vkmaljyanc:~$ rm /tmp/file01.txt
rm: remove write-protected regular file '/tmp/file01.txt'? y
guest2@vkmaljyanc:~$ ls -l / | grep tmp
drwxrwxrwx. 14 root root 4096 Apr  2 19:41 tmp
guest2@vkmaljyanc:~$ █

```

Рисунок 3.34: Повторение предыдущих шагов

Повышаю свои права до суперпользователя и возвращаю атрибут `t` на директорию `/tmp`: `su - chmod +t /tmp exit` (рис. 3.35) [lab05].

```

guest2@vkmaljyanc:~$ su -
Password:
Last login: Thu Apr  2 19:39:22 MSK 2026 on pts/6
root@vkmaljyanc:~# chmod +t /tmp
root@vkmaljyanc:~# exit
logout
guest2@vkmaljyanc:~$

```

Рисунок 3.35: Применение команд `su -`, `chmod +t /tmp`, `exit`

4 Выводы

Я изучила механизмы изменения идентификаторов, применения SetUID- и Sticky-битов. Получила практические навыки работы в консоли с дополнительными атрибутами. Рассмотрела работу механизма смены идентификатора процессов пользователей, а также влияние бита Sticky на запись и удаление файлов.

Список литературы